

第25回 クラスを整理する:packageとAPI

今回のゴール

- Pleiades でパッケージを作り、クラスをパッケージに入れて実行できる。
- `package` 宣言とフォルダの関係を説明できる。
- `Random` を使って、敵がねらう相手をランダムに決められる。
- `static` の変数・メソッド・定数が「クラスに 1 つだけ」あることを説明できる。

前回とのつながり

第24回で、クラスのファイルが 7 つになった。今回は、バトル用のクラスを `battle` という パッケージ にまとめる。また、第24回まで敵がねらう相手は `turn % party.size()` で決まっていたので、毎回同じ展開だった。今回は 乱数 を使って、毎回違うバトルにする。

実習時のフォルダ構成

今回から、クラスを `battle` パッケージに入れる。パッケージは、フォルダとして表示される。

```
ワークスペース
├─ java25_sample
│   └─ src
│       └─ battle      ← パッケージ
│           ├─ Character.java
│           ├─ Hero.java
│           ├─ Wizard.java
│           ├─ Warrior.java
│           ├─ Priest.java
│           ├─ Enemy.java
│           └─ Main.java
└─ java25_exercise
    └─ src
        └─ battle
            └─ (sample と同じ 7 ファイル)
```

- 配布ファイル: プログラム/第25回/sample/battle/、プログラム/第25回/exercise/battle/
- 解答例: プログラム/第25回/answer/battle/

Pleiades でパッケージを作る

1. `java25_sample` プロジェクトを作る(第16回と同じ)。
2. `src` を右クリックし、「新規」→「パッケージ」を選ぶ。名前に `battle` と入力して「完了」を押す。
3. `battle` パッケージを右クリックし、「新規」→「クラス」でクラスを作る。パッケージの欄に `battle` と入っていることを確認する。

4. 作ったクラスの 1 行目に、`package battle;` が自動で書かれる。配布ファイルの内容を貼り付けるときは、この行を二重に書かないように注意する。

講義内容

1. package:クラスのグループ

```
package battle;

public class Hero extends Character {
    ...
}
```

- ファイルの 1 行目の `package battle;` は、「このクラスは `battle` パッケージに入っている」という宣言。
- パッケージ名とフォルダ名は同じにする。`battle` パッケージのクラスは、`src/battle/` フォルダに置く。
- 同じパッケージのクラスどうしは、`import` なしで使える。
- 第24回で学んだ `java.util` も、Java が用意しているパッケージの 1 つ。`import java.util.Random;` は「`java.util` パッケージの `Random` クラスを使う」という意味。
- これまで作っていたクラスは、パッケージのない「デフォルト・パッケージ」に入っていた。第16回で出た警告は、このことだった。

補足:Character という名前

Java には、`java.lang` パッケージに `Character` (1 文字を扱うクラス)が最初から用意されている。`battle` パッケージの中では、自分で作った `battle.Character` が優先して使われるので、このバトルでは問題ない。ほかのプログラムで `Character` と書いたときは、別のクラスを指すことがある、とだけ覚えておく。

2. Random:乱数を使う

```
import java.util.Random;

Random random = new Random();
int n = random.nextInt(3);    // 0、1、2 のどれか
```

- `nextInt(数)` は、0 から「数 - 1」までの整数を、ランダムに 1 つ返す。
- `party.get(random.nextInt(party.size()))` で、パーティーの中から 1 人をランダムに選べる。人数が 3 人なら、0～2 番のどれかになる。

3. static:クラスに 1 つだけあるもの

プログラム/第25回/sample/battle/Character.java (変わったところ)

```
// static 変数: インスタンスごとではなく、クラスに 1 つだけある (全員で共有する)
private static int count = 0;

public Character(String name, int hp, int power) {
    ...
    count++; // キャラクターが 1 人作られるたびに数える
}

// static メソッド: インスタンスがなくても Character.getCount() で呼べる
public static int getCount() {
    return count;
}
```

	ふつうのフィールド(hp など)	static 変数(count)
いくつあるか	インスタンスごとに 1 つ	クラスに 1 つだけ
使い方	hero.getHp()	Character.getCount()
例	勇者の HP、スライムの HP	作られたキャラクターの合計人数

- 第18回で、main から呼ぶメソッドに static を付けた理由はこれ。main は、インスタンスを作らずに最初
に実行される static メソッドなので、同じクラスのメソッドを直接呼ぶには、そのメソッドも static にす
る必要があった。
- static final を付けると、全員で共有する「変わらない値」(定数)になる。定数の名前は HERB_POWER の
ように大文字と _ で書く習慣がある。

```
public static final int HERB_POWER = 15;
```

4. Math:計算用の API

Character.java の HP の計算を、Math クラスのメソッドで短く書き直した。

```
hp = Math.max(hp - amount, 0); // 2 つのうち大きいほう: 0 より小さくならない
hp = Math.min(hp + amount, maxHp); // 2 つのうち小さいほう: maxHp より大きくならない
```

- Math は java.lang パッケージのクラスなので、import なしで使える。static メソッドなので、new せ
ずに Math.max(...) と呼ぶ。
- 第20回の if で書いた処理と、結果は同じ。

5. サンプルを読む

プログラム/第25回/sample/battle/Main.java (変わったところ)

```
package battle;

import java.util.ArrayList;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Random random = new Random();    // 乱数 (ランダムな数) を作る道具
        ...
        System.out.println("魔物の群れがあらわれた！ (登場キャラクター：" + Character.getCount() +
            "人) ");
        ...
        // 残っている敵が、1 体ずつ攻撃する
        for (Enemy enemy : enemies) {
            if (party.isEmpty()) {
                break;
            }
            // ねらう相手をランダムに決める (0 ~ 人数-1 のどれか)
            Character target = party.get(random.nextInt(party.size()));
            enemy.attack(target, turn);
            ...
        }
    }
}
```

- 4 ターン目にスライムが増える処理(第24回の演習)も入っている。

6. 実行する前に予想する

- 最初に表示される「登場キャラクター」は何人か(new している数を数える。4 ターン目のスライムはまだ作られていない)。
- 2 回実行したとき、結果は同じになるか。

7. 実行結果の例

最初の部分は、毎回同じになる。

```
魔物の群れがあらわれた！（登場キャラクター：7人）

--- ターン1 ---
勇者 HP:30/30
魔法使い HP:15/15
戦士 HP:40/40
敵：3体
スライム HP:20/20
ゴブリン HP:40/40
オーク HP:70/70
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ スライムに8のダメージ
魔法使いは火の魔法をとねえた！ スライムに10のダメージ
戦士のおのの一撃！ スライムに10のダメージ
スライムをたおした！
```

敵の攻撃は、実行するたびに変わる(例)。

```
ゴブリンの攻撃！ 戦士に7のダメージ
オークの攻撃！ 勇者に9のダメージ
```

- 何回か実行して、ねらわれる相手が変わることを確かめる。

8. エラーを体験する

1. `Hero.java` の 1 行目を `package game;` に変えて保存する。
2. 「宣言されたパッケージが、予期されるパッケージと一致しない」というエラーになる。`Hero.java` は `battle` フォルダにあるのに、`game` パッケージだと宣言したため。
3. 元に戻す。次に `Main.java` の `import java.util.Random;` を消して、`Random` が見つからないエラーを確認する。確認したら元に戻す。

演習1 サンプルを実行する

`java25_sample` を作り、`battle` パッケージにファイルを入れて実行する。2～3 回実行して、毎回結果が変わることを確認する。

演習2 パッケージと乱数

`java25_exercise` を作り、`battle` パッケージに `プログラム/第25回/exercise/battle/` の 7 ファイルを入れる。

演習2-1 パッケージの確認

- すべてのファイルの 1 行目が `package battle;` になっていること、Pleiades で `src/battle` の中に 7 ファイルがあることを確認して実行する。

演習2-2 敵がランダムにねらう

- 今は `turn % party.size()` で、ねらう相手が毎回同じ順番になっている。
- `Random` を `import` し、`main` の最初で `Random random = new Random();` と用意する。
- ねらう相手を、`random.nextInt(party.size())` 番目のメンバーにする。

演習2-3 薬草の回復量を定数にする

- `Hero` クラスに、`public static final int HERB_POWER = 15;` を追加する。
- `useHerb` の中の `heal(15)` を `heal(HERB_POWER)` に変える。
- `Main` の最初に `System.out.println("薬草の回復量：" + Hero.HERB_POWER);` と書いて、インスタンスがなくてもクラス名で使えることを確認する。

完成条件:

- 2 回以上実行して、敵の攻撃の相手が変わる。
- 薬草を使ったとき、今までどおり 15 回復する。

(乱数のため、実行結果は毎回変わる)

発展(任意)

- 勇者の会心の一撃を「3 ターンごと」から「4 回に 1 回くらいの確率(`random.nextInt(4) == 0`)」に変える。`Hero` の中で `Random` を使うには、`Hero.java` にも `import` が必要になる。
- 敵の攻撃力に、毎回 0~2 の乱数を足す。

確認問題

問1 出力は何か。

```
class Coin {
    static int total = 0;
    int mine = 0;

    Coin() {
        total++;
        mine++;
    }
}
```

```
Coin a = new Coin();
Coin b = new Coin();
System.out.println(Coin.total + " " + b.mine);
```

問2 出力は何か。

```
System.out.println(Math.max(3, 8) + " " + Math.min(10, 4));
```

問3 `random.nextInt(3)` が返す可能性のある値を、すべて答える。

■ 答え

- 問1: 2 1 (`total` は全員で 1 つなので 2。 `mine` はインスタンスごとなので、 `b` の `mine` は 1)
- 問2: 8 4
- 問3: 0、1、2 (3 は含まない)

振り返り

- `package` 宣言とフォルダの関係を説明できるか。
- `import` が必要なクラス (`Random`、`ArrayList`) と、必要ないクラス (`String`、`Math`、同じパッケージのクラス) を区別できるか。
- `static` 変数と、ふつうのフィールドの違いを説明できるか。

次回へのつながり

プログラムが大きくなり、書き間違いやエラーも起きやすくなった。第17回からの約束だった「数字以外を入力すると止まる」問題も残っている。次回は、いろいろな種類のエラーを読んで直す練習をする。